

CachyOS Secure Boot + TPM2 Auto-Unlock Guide (Limine)

Tested in QEMU VM before applying to physical hardware.

Bootloader: Limine (not GRUB - GRUB 2.14 has Secure Boot regressions)

GPU: NVIDIA - use `nvidia` package (not `nvidia-dkms`) for Secure Boot compatibility

Part 1: VM Setup for Testing

Prerequisites

```
sudo pacman -S qemu-full swtpm edk2-ovmf
```

Script Usage

```
# Non-Secure Boot (install CachyOS here)
./setup-test-vm.sh launch ~/Downloads/CachyOS.iso
```

```
# Secure Boot (for testing Secure Boot setup)
./setup-test-vm.sh launch --secure-boot
```

```
# Destroy and start fresh
./setup-test-vm.sh destroy
```

Important Notes

- **Non-Secure Boot** uses `pc` machine type with `OVMF_CODE.4m.fd` + `OVMF_VARS.4m.fd`
 - **Secure Boot** uses `q35` machine type with `OVMF_CODE.secboot.4m.fd` + `OVMF_VARS.secboot.4m.fd`
 - Each mode has its own VARS file, so enrolled keys are preserved when switching between modes
 - Use **F2** or **Delete** to enter UEFI firmware settings (not Escape - that closes the menu)
-

Part 2: Secure Boot Setup (inside CachyOS VM - Limine)

Step 0: Install Required Packages

```
sudo pacman -S sbctl mokutil limine-mkinitcpio-hook
```

- `sbctl` - Secure Boot key management (create, enroll, sign)
- `mokutil` - verify Secure Boot state

- `limine-mkinitcpio-hook` - signs Limine EFI binary on kernel updates

NVIDIA users: Install `nvidia` (not `nvidia-dkms`). DKMS modules can't load under `lockdown=integrity` on CachyOS.

Step 1: Enter Setup Mode

```
sudo systemctl reboot --firmware-setup
```

In UEFI/BIOS: 1. Find **Secure Boot** settings 2. Set Secure Boot Mode to **Custom** 3. Go to Key Management -> **Delete all Secure Boot Variables** 4. Save & Exit

Step 2: Create and Enroll Keys

```
# Create Secure Boot keys
```

```
sudo sbctl create-keys
```

```
# Enroll keys (includes Microsoft keys for Limine)
```

```
sudo sbctl enroll-keys --microsoft
```

Note: Do NOT use `--firmware-builtin` - it doesn't work in QEMU VMs.

Step 3: Enroll Limine Config and Sign

```
# Add enrollment setting if not already present
```

```
sudo bash -c 'grep -q "ENABLE_ENROLL_LIMINE_CONFIG" /etc/default/limine 2>/dev/null || echo
```

```
# Enroll config checksum and sign Limine EFI binary (does both in one step)
```

```
sudo limine-enroll-config
```

```
sudo limine-update
```

Important: - Do NOT use `sbctl-batch-sign` with Limine - it detects Limine and refuses to run - Do NOT run `sbctl sign` separately - `limine-enroll-config` signs the binary using `sbctl` under the hood - `sbctl verify` will show unsigned files in `limine_history/` - this is normal, ignore those - **Splash background:** When Secure Boot is active, Limine verifies the splash image hash. If the splash wasn't included in enrollment, it falls back to the default background. This is normal and means Secure Boot is working. To keep your custom splash, include it in the enrollment (see Limine docs).

Step 5: Enable Secure Boot

```
sudo systemctl reboot --firmware-setup
```

In UEFI/BIOS: 1. Set Secure Boot Mode to **Standard** (or **Enabled**) 2. Save & Exit

Step 6: Verify Secure Boot

```
mokutil --sb-state
# Should say: SecureBoot enabled

sudo sbctl status
# Should show: Secure Boot: Enabled
```

Part 3: TPM2 Auto-Unlock Setup (inside CachyOS)

Step 1: Verify Prerequisites

```
# Install required packages (systemd-cryptenroll is part of systemd, already installed)
sudo pacman -S tpm2-tss tpm2-tools

# Check TPM2 is available
systemd-analyze has-tpm2
# Expected: yes

# Check LUKS version (must be LUKS2)
sudo cryptsetup luksDump /dev/nvmeXnXpX | grep Version
# Expected: Version: 2

# Find your LUKS partition
lsblk -f
```

Step 2: Create crypttab.initramfs

```
sudo cp /etc/crypttab /etc/crypttab.initramfs
```

Verify the file - your root partition should have none as the password:

```
luks-XXXXXXXXXXXX UUID=XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX none tpm2-device=auto
```

Step 3: Enroll TPM2

```
# Enroll with TPM2 (adjust /dev/nvmeXnXpX to your LUKS partition)
# PCR 7 = Secure Boot state (unlocks only if boot chain hasn't changed)
sudo systemd-cryptenroll --wipe-slot tpm2 \
    --tpm2-device auto \
    --tpm2-pcrs "7" \
    /dev/nvmeXnXpX

# Create a recovery key (SAVE THIS SOMEWHERE SAFE OFFLINE)
sudo systemd-cryptenroll --recovery-key /dev/nvmeXnXpX
```

Note: `--wipe-slot tpm2` only wipes TPM2 keyslots on the partition you specify. It does NOT affect other drives or Windows BitLocker. The TPM chip stores multiple independent keys - wiping a LUKS TPM slot doesn't touch BitLocker's TPM binding. Safe for dual-boot.

PCR Selection Guide:

PCR	What it measures	Recommendation
0	UEFI firmware	Skip (changes on firmware updates)
1	Hardware config	Include (detects hardware changes)
2	Option ROMs	Skip
3	Option ROM config	Include
4	Boot loader	Skip (changes on kernel updates)
5	GPT partition table	Include
6	Hibernate resume	Skip
7	Secure Boot state	Include (core requirement)
11	Kernel image	Include
12	Kernel config	Include
14	Shim	Include

Recommended PCR set: "1+3+5+7+11+12+14"

Dual-boot note: PCR 7 (Secure Boot state) is shared. If you enable/disable Secure Boot, BOTH CachyOS TPM auto-unlock and Windows BitLocker will ask for password/recovery key on next boot. This is expected - the boot chain changed. Re-enroll after changing Secure Boot state.

Step 4: Update initramfs Hooks

Edit `/etc/mkinitcpio.conf`:

```
sudo nano /etc/mkinitcpio.conf
```

Find the `HOOKS` line and change from:

```
HOOKS=(base udev autodetect microcode kms modconf block keyboard keymap consolefont plymouth)
```

To:

```
HOOKS=(base systemd plymouth autodetect microcode modconf kms keyboard sd-vconsole sd-encrypt)
```

Changes: - `udev` -> `systemd` - `keymap consolefont` -> removed (handled by `sd-vconsole`) - `encrypt` -> `sd-encrypt` - Added `sd-vconsole` and `fsck`

Step 5: Rebuild initramfs

```
sudo mkinitcpio -P
```

Step 6: Test

`sudo reboot`

Expected behavior: - VM boots without asking for LUKS password - If boot chain changes (different kernel, Secure Boot disabled), TPM refuses and you use the recovery key

Part 4: Troubleshooting

“Display output is not active”

- **Cause:** Wrong VGA adapter or machine type
- **Fix:** Use `-vga std` (not `-vga virtio`) and `q35` machine type for Secure Boot

“Guest has not initialized the display (yet)”

- **Cause:** Secure Boot OVMF not loading properly
- **Fix:** Ensure using `q35` machine type with `pflash`, not `-bios`

“System is not booted with UEFI” (`sbctl`)

- **Cause:** Using `-bios` flag instead of `pflash`, or non-Secure-Boot OVMF
- **Fix:** Use `pflash` with `OVMF_CODE.secboot.4m.fd` + `OVMF_VARS.4m.fd` on `q35`

PXE boot instead of OS

- **Cause:** OVMF VARS file corrupted or missing
- **Fix:** Delete `OVMF_VARS.4m.fd` from VM directory, let script recreate it

“access denied failed to load Boot00A”

- **Cause:** Stale boot entries in OVMF VARS from switching between Secure Boot modes. The VARS file stores UEFI boot entries (like `Boot00A`) that were created in one mode - switching to the other mode makes them invalid.
- **Fix:** The script now uses separate VARS files for each mode (`OVMF_VARS.4m.fd` for non-secure boot, `OVMF_VARS.secboot.4m.fd` for secure boot), so enrolled keys are preserved when switching. If you still get this error, reset VARS with `--reset-vvars`:

```
./setup-test-vm.sh launch --secure-boot --reset-vvars
```

Then re-enroll Secure Boot keys and TPM inside the VM (data on disks is safe).

“couldn’t sync keys: open /sys/firmware/efi/efivars/dbDefault”

- **Cause:** `--firmware-builtin` flag in QEMU
- **Fix:** Use `sudo sbctl enroll-keys --microsoft` (without `--firmware-builtin`)

“not authorized” on Secure Boot boot

- **Cause:** Secure Boot was enabled in firmware BEFORE keys were enrolled. The signed Limine binary isn’t signed by Microsoft’s default keys, so Secure Boot rejects it.
- **Fix:** You must enter Setup Mode (delete all Secure Boot variables in UEFI) BEFORE running `sbctl create-keys`. Boot with `--secure-boot`, press Escape at the CachyOS logo to enter UEFI, delete all Secure Boot variables, save & exit, then boot into CachyOS and enroll keys.

“sbctl-batch-sign: Limine detected, please do not use this script”

- **Cause:** `sbctl-batch-sign` doesn’t work with Limine
- **Fix:** Don’t use it. Limine only needs `limine-enroll-config` + `limine-update` + manual signing of `limine_x64.efi`

“sbctl verify” shows unsigned files in `limine_history/`

- **Cause:** Normal - `limine_history/` contains old kernel checksums, not boot-critical files
- **Fix:** Ignore. Only `limine_x64.efi` and kernel images in `/boot/` need signing

Limine checksum verification fails after signing

- **Cause:** `sbctl sign` modifies files, breaking Limine’s checksums
- **Fix:** Set `ENABLE_VERIFICATION=no` in `/etc/default/limine` and run `sudo limine-update`, OR use `limine-mkinitcpio-hook` which handles signing before checksums are computed

NVIDIA driver not loading with Secure Boot

- **Cause:** DKMS modules can’t load under `lockdown=integrity` on CachyOS (`CONFIG_IMA` disabled)
- **Fix:** Use `nvidia` package (not `nvidia-dkms`), or add `lockdown=none` to kernel parameters

TPM auto-unlock fails after kernel update

- **Cause:** PCR values changed (if including PCR 4/9)
- **Fix:** Re-enroll TPM2 with updated kernel:

```
sudo systemd-cryptenroll --wipe-slot tpm2 --tpm2-device auto --tpm2-pcrs "7" /dev/nvmeX
```

Emergency: Use recovery key

If TPM refuses to unlock: 1. At the LUKS prompt, enter your recovery key 2. Once booted, re-enroll TPM2

Quick Reference: Full Procedure (VM - Limine)

VARs files: The script uses separate VARs files per mode: - OVMF_VARS.4m.fd - non-Secure Boot (no enrolled keys) - OVMF_VARS.secboot.4m.fd - Secure Boot (enrolled keys live here)

Each mode's keys are independent. Switching modes preserves both.

1. Destroy and reinstall

```
./setup-test-vm.sh destroy
./setup-test-vm.sh launch ~/Downloads/CachyOS.iso
```

2. Install CachyOS, shut down

3. Boot with Secure Boot

```
./setup-test-vm.sh launch --secure-boot
```

4. At boot screen - press Escape to enter UEFI settings

Delete all Secure Boot variables (Setup Mode) -> Save & Exit

VM reboots into CachyOS (Secure Boot is now off)

5. Inside CachyOS VM - enroll keys

```
sudo pacman -S sbctl mokutil limine-mkinitcpio-hook
```

```
sudo sbctl create-keys
```

```
sudo sbctl enroll-keys --microsoft
```

Enable Limine config checksum enrollment and sign

```
sudo bash -c 'grep -q "ENABLE_ENROLL_LIMINE_CONFIG" /etc/default/limine 2>/dev/null || echo
```

```
sudo limine-enroll-config
```

```
sudo limine-update
```

```
sudo sbctl status
```

6. Reboot into UEFI to enable Secure Boot

```
sudo systemctl reboot --firmware-setup
```

If that doesn't work in QEMU, shut down and relaunch:

sudo shutdown now

./setup-test-vm.sh launch --secure-boot

Then press Escape at boot screen to enter UEFI

In UEFI: set Secure Boot to Standard -> Save & Exit

7. Verify

```
mokutil --sb-state
```

8. TPM2 Auto-Unlock

```
sudo pacman -S tpm2-tss tpm2-tools
```

```
sudo cp /etc/crypttab /etc/crypttab.initramfs
```

```
sudo systemd-cryptenroll --wipe-slot tpm2 --tpm2-device auto --tpm2-pcrs "7" /dev/nvmeXnXpX
```

```
sudo systemd-cryptenroll --recovery-key /dev/nvmeXnXpX
```

9. Update initramfs hooks

```
sudo nano /etc/mkinitcpio.conf
```

Change HOOKS: udev->systemd, encrypt->sd-encrypt, add sd-vconsole

```
sudo mkinitcpio -P
```

10. Reboot and test

```
sudo reboot
```

Critical: Step 4 (entering UEFI at boot screen) must happen BEFORE step 5. If Secure Boot is already enabled when you enroll keys, the signed binary won't match the firmware's expectations and you get "not authorized."